

Portage du Bytestream Decoder sur Coyote

**Master of Science HES SO in
Engineering**

Profil/Orientation Computer Science

Date of publication: 31.05.2026

Produced by:

Author Abivarman Kandiah

Under the supervision of Andres Upegui Posada, Quentin Berthet



Table of Contents

Declaration of authenticity	3
Acknowledgments	4
Resume (English version)	5
Résumé (version française)	5
1. Context (Résumé Temporaire)	6
2. Introduction	6
3. Contexte technique	6
3.1. Accélération matérielle par FPGA pour le traitement de flux	6
3.2. Frameworks de développement hôte-FPGA : XRT et Coyote	7
3.3. Le <i>Bytestream Decoder</i> comme algorithme de référence	8
4. Prise en main et évaluation de Coyote	8
4.1. Mise en place de l'environnement	8
4.2. Tests sur U55C — difficultés rencontrées	8
4.3. Tests sur V80 — stabilisation	9
4.4. Retour d'expérience sur Coyote comme framework	9
5. Portage du Bytestream Decoder sur Coyote	9
5.1. Adaptation du wrapper hardware (Design 1)	9
5.2. Adaptation du code hôte	9
6. Analyse des performances et optimisation	9
6.1. Méthodologie de mesure (ILA)	9
6.2. Design 1 — diagnostic du goulot	10
6.3. Design 2 — sortie 128 bits	10
6.4. Montée en fréquence à 400 MHz	10
6.5. Tableau comparatif et bilan	10
7. Discussion : Coyote face à XRT et perspectives smartNIC	10
7.1. Coyote vs XRT — bilan du portage	10
7.2. Pertinence pour des cas d'usage de type smartNIC	10
7.3. Perspectives d'amélioration	11
8. Conclusion	11
Bibliography	12
A Code — Wrapper vfpga_top.svh	14
B Script d'analyse ILA	14





Declaration of authenticity

I hereby affirm that this assignment is my own written work and that I have used no other sources or aids other than those indicated. All passages that have been quoted from publications or paraphrased from these sources are indicated as such. Additionally, any assistance from artificial intelligence tools has been appropriately acknowledged and documented.

Date : 31.05.2026

Name : Abivarman Kandiah





Acknowledgments

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequale doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.



structured-mse-thesis.typ	To be finalized	
HES-SO / Domaine I&A / MSE / Travail de Master	31-05-2026 - Abivarman Kandiah	- 4 / 14 -



Resume (English version)

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Résumé (version française)

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.



également de la HBM [2]. Au démarrage du projet, le support de la V80 par les frameworks logiciels existants était quasi inexistant, ce qui en faisait précisément une plateforme intéressante à évaluer.

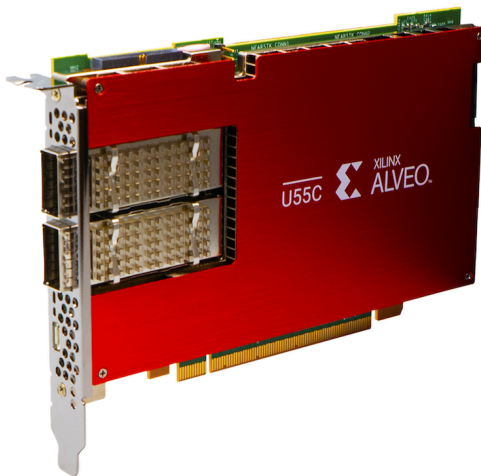


Figure 1: Xilinx Alveo U55C, une carte d'accélération basée sur un FPGA UltraScale+.

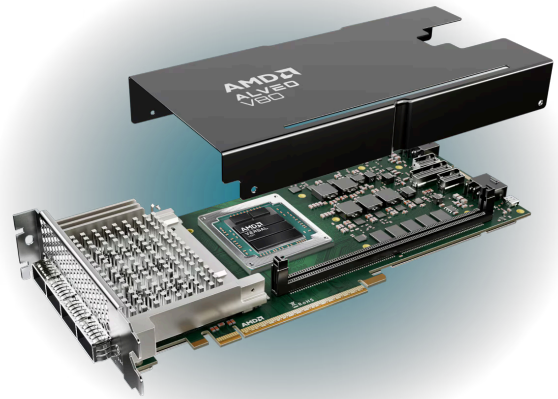


Figure 2: Xilinx Versal V80, une carte d'accélération basée sur un FPGA Versal.

3.2. Frameworks de développement hôte-FPGA : XRT et Coyote

Exploiter un FPGA depuis un programme hôte suppose une pile logicielle qui prend en charge le transfert PCIe, la gestion mémoire, l'orchestration des kernels et la synchronisation. Deux frameworks sont ici comparés.

XRT (*Xilinx Runtime*) est la pile officielle d'AMD/Xilinx pour les cartes Alveo [3]. Le modèle de programmation repose sur des *kernels* — généralement décrits en Vitis HLS, parfois en RTL — empaquetés dans un binaire `.xclbin` chargé par l'hôte. Les données transitent via des *buffers* explicitement alloués, et l'API C++ (OpenCL ou XRT native) suit un schéma *offload batch* : copie hôte→FPGA, exécution, copie FPGA→hôte. XRT bénéficie d'un écosystème mature, étroitement intégré à la *toolchain* Vitis, mais présente deux limites pour le présent projet : il ne supporte pas la Versal V80, et son modèle batch s'adapte assez mal aux pipelines de flux continus typiques des cas d'usage *dataplane*.

Coyote est un framework open source développé à l'ETH Zürich qui adopte une approche radicalement différente [4]. L'idée centrale est une séparation stricte *shell/role* : le *shell* regroupe toute l'infrastructure générique (contrôleur PCIe, DMA, gestion mémoire, interruptions, hugepages, support multi-locataire et RDMA optionnel) et est livré pré-implémenté ; le *role* correspond à la logique applicative écrite par l'utilisateur, qui dialogue avec le shell via une ou plusieurs interfaces AXI-Stream de 512 bits. Le driver noyau associé est générique et rechargeable sans reflasher le FPGA. Cette architecture cible explicitement les déploiements *dataplane* et *smartNIC*, et le support de cartes récentes — notamment la V80 — y est ajouté plus rapidement que dans XRT. En contrepartie, la documentation est plus parcellaire et la maturité de l'outil reste inférieure à celle de XRT.

Le tableau Table 1 synthétise les principales différences.

	XRT	Coyote
Modèle	Offload batch	Pipeline shell/role
Interface vers le <i>role</i>	Buffers + kernels	AXI-Stream 512 bits
Support V80	Non	Oui
Toolchain requise	Vitis	Vivado seul
Maturité	Élevée	En cours
Cible privilégiée	Calcul accéléré	Dataplane / smartNIC

Table 1: Comparaison synthétique de XRT et Coyote.

3.3. Le *Bytestream Decoder* comme algorithme de référence

L'algorithme retenu pour évaluer Coyote sur un cas réaliste est le *Bytestream Decoder*, un décodeur initialement développé dans le cadre de l'expérience ATLAS au CERN, plus précisément pour la chaîne de lecture du calorimètre à argon liquide (LAr) [5]. Sans entrer dans la physique sous-jacente, il suffit de retenir que les cartes *Front-End Boards* (FEB) du détecteur émettent à chaque collision un flux compressé décrivant l'énergie déposée dans chaque cellule du calorimètre ; le décodeur reconstruit, à la volée, une représentation structurée de ces cellules.

Ce choix est doublement pertinent. D'une part, l'algorithme constitue un cas d'usage représentatif d'une large famille de traitements *dataplane* : décompression et parsing à la volée, consultation de tables (LUT), correction en arithmétique flottante, fusion de plusieurs flux d'entrée. D'autre part, le décodeur existe déjà sous la forme d'un design VHDL validé sous XRT [6] sur Alveo U55C, ce qui en fait un point de comparaison idéal pour mesurer le coût et les bénéfices d'un portage vers Coyote.

Le pipeline VHDL s'organise en plusieurs étages successifs. Un *FEB Parser* extrait les en-têtes et les champs utiles du flux brut, puis des décodeurs spécialisés reconstituent le gain, l'énergie et le temps de chaque cellule. Une table CAM associée à une LUT hachée fournit les coefficients de correction, qui sont appliqués par une unité flottante (FPU) avant qu'un *Output Merger* ne sérialise le résultat dans une FIFO de sortie. En interne, le bus est large de 32 bits ; le module expose deux flux AXI-Stream en entrée (données brutes et corrections initiales) et produit en sortie une structure de quatre mots de 32 bits par cellule (*Gain+ID*, *Energy*, *Time*, *Quality*). Cette largeur de 32 bits, héritée du contexte XRT, jouera un rôle central dans l'analyse de performance présentée au chapitre 5.

4. Prise en main et évaluation de Coyote

4.1. Mise en place de l'environnement

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

4.2. Tests sur U55C — difficultés rencontrées

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.



idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.

4.3. Tests sur V80 — stabilisation

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.

4.4. Retour d'expérience sur Coyote comme framework

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.

5. Portage du Bytestream Decoder sur Coyote

5.1. Adaptation du wrapper hardware (Design 1)

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.

5.2. Adaptation du code hôte

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et.

6. Analyse des performances et optimisation

6.1. Méthodologie de mesure (ILA)

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.



6.2. Design 1 — diagnostic du goulot

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

6.3. Design 2 — sortie 128 bits

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.

6.4. Montée en fréquence à 400 MHz

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

6.5. Tableau comparatif et bilan

Design	Fréquence	Sortie	Temps transfert	tready entrée
Design 1	250 MHz	32 bits	3.17 ms	20.9 %
Design 2	250 MHz	128 bits	0.83 ms	88.7 %
Design 2	400 MHz	128 bits	0.82 ms	90.0 %

Table 2: Comparaison des performances des trois configurations testées sur V80.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri.

7. Discussion : Coyote face à XRT et perspectives smartNIC

7.1. Coyote vs XRT — bilan du portage

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.

7.2. Pertinence pour des cas d'usage de type smartNIC

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod



idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.

7.3. Perspectives d'amélioration

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim admodum doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere.

8. Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim admodum doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguere possit, augeri amplificarique non possit. At.



Bibliography

- [1] AMD/Xilinx, “Alveo U55C High Performance Compute Card – Product Brief.” 2022.
- [2] AMD, “AMD Alveo V80 Compute Accelerator Card – Product Brief.” 2024.
- [3] AMD/Xilinx, “XRT: Xilinx Runtime Library Documentation.” 2024.
- [4] D. Korolija, T. Roscoe, and G. Alonso, “Do OS Abstractions Make Sense on FPGAs?,” in *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, USENIX Association, 2020. [Online]. Available: <https://www.usenix.org/conference/osdi20/presentation/roscoe>
- [5] ATLAS LAr Collaboration, “The ATLAS Liquid Argon Calorimeter: Construction, Integration, Commissioning and Performance,” *Journal of Instrumentation*, vol. 5, p. P9003, 2010.
- [6] A. Upegui Posada, “Bytestream Decoder – design VHDL pour ATLAS LAr sur Alveo U55C.” 2024.
- [7] D. Firestone and others, “Azure Accelerated Networking: SmartNICs in the Public Cloud,” in *15th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, USENIX Association, 2018.



Appendices

A Code — Wrapper vfpga_top.svh	14
B Script d'analyse ILA	14





A Code — Wrapper vfpga_top.svh

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri.

B Script d'analyse ILA

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aequi doleamus animo, cum corpore dolemus, fieri.

